

# Cloud Engineering Accelerator, Lab 01

## Hosting Your First Static Website in Azure

A step by step Azure lab, written so a 12 year old can follow along. Built entirely with the Azure CLI inside VS Code, no clicking through the Azure Portal required.

| Quick Facts                                                                                          |
|------------------------------------------------------------------------------------------------------|
| What you're building: a public website served directly from Azure Blob Storage, no web server needed |
| Resource Group: rg-lab01-yourname                                                                    |
| Location: East US                                                                                    |
| Deploy time: about 10 minutes                                                                        |
| Cost: well under \$5 a month for realistic personal traffic, full cost breakdown further down        |

**NOTE:** This SOP is self-contained. Every command and every file's content is printed in full below. The original accelerator material was written as portal click-through steps. This version replaces every click with an Azure CLI command, so the whole lab is scriptable, repeatable, and runs entirely inside VS Code's terminal instead.

## What This Lab Covers and Why It Matters

Most websites need a server running somewhere, listening for requests, staying patched, staying online. This lab skips all of that. Azure Blob Storage can serve plain HTML, CSS, and JavaScript files directly to the public internet on its own, with no virtual machine, no web server software, and no patching ever required. This is called serverless hosting, and the underlying model is PaaS, Platform as a Service, meaning Azure runs the infrastructure and you only manage the content.

This pattern is genuinely used in production for real portfolio sites, documentation sites, and marketing pages, not just as a teaching example. The same storage account technique scales from a single personal page to a high traffic public site when paired with a content delivery network, which this SOP also covers as an optional upgrade.

## What You Will Learn

| Skill                               | Why it matters in a real environment                                                                                                                                 |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serverless static hosting           | Understanding when you genuinely don't need a server is a real cost and security win. Fewer moving parts means fewer things to patch and fewer ways to get breached. |
| Azure CLI resource creation         | Clicking through a portal doesn't scale and isn't repeatable. Real teams script their infrastructure so it can be rebuilt identically at any time.                   |
| Storage account security settings   | HTTPS enforcement, TLS version, and public access scope are the exact settings that separate a secure public site from an accidentally wide open storage account.    |
| Reading a real Azure cost breakdown | Knowing what actually drives a bill, storage, transactions, and egress, is what lets you predict cost before you deploy, not after a surprise invoice.               |

## Architecture

A visitor's browser sends a request over HTTPS straight to your storage account's public website endpoint. Azure Storage reads the requested file, in this case index.html, out of a special container named \$web, and returns it directly. There is no server in between, no virtual machine, and nothing for you to patch or reboot.

| Piece                                                       |
|-------------------------------------------------------------|
| Visitor's browser                                           |
| HTTPS request to your storage account's public web endpoint |
| Azure Storage Account, the \$web container specifically     |

index.html, the file actually returned to the visitor

## Before You Begin: Confirm Your Starting Path

**NOTE:** Your working folder for this lab: C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\static-website-lab. Every command below is already written for that exact location.

### Confirm your terminal location

```
# See exactly where you are right now
Get-Location
```

If that does not show your CEA LABS folder, navigate there first:

```
cd "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS"
Get-Location # should now show the CEA LABS folder
```

### Check whether this lab's folder already exists

```
Test-Path "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\static-website-lab"
```

| Result | What to do                                                                                                                                                                                                                                                                                                                               |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| False  | It doesn't exist yet. Continue to Step 1 and create it.                                                                                                                                                                                                                                                                                  |
| True   | It already exists from an earlier attempt. Skip the New-Item commands in Step 1. Just run: cd "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\static-website-lab" then code "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\static-website-lab" to open VS Code pointed directly at it |

## Prerequisites: Verify Before Starting

```
az version # Any recent Azure CLI version
```

Sign in using your tenant ID and device code, this method avoids the browser sometimes routing to the wrong Microsoft sign-in page, a real problem encountered earlier in this series:

```
az login --tenant aa8b6d7b-653e-4fb2-9fca-fee0d0feff60 --use-device-code
```

This prints a one-time code and normally opens your browser automatically to a Microsoft sign-in page with that code already filled in.

**NOTE:** If a browser window doesn't open on its own, don't wait on it. Open one yourself and go directly to <https://microsoft.com/devicelogin>, then type in the code the terminal printed. Use a fresh incognito or InPrivate window for this, that avoids any cached sign-in session interfering with which account it tries to use.

Confirm you're pointed at the right subscription:

```
az account show
```

## Running This Lab in VS Code

Same setup as the rest of the series. If you already have the Azure Account extension installed, skip straight to opening the folder below.

| Extension     | What it's for                                                                                                           |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| Azure Account | Lets VS Code share the same az login session as your terminal                                                           |
| Azure Storage | Optional. Lets you browse your storage account's containers directly inside VS Code, useful for double checking uploads |

## Open VS Code to your CEA LABS folder first

**NOTE:** This lab's project folder doesn't exist yet on a first run, so don't try to open it directly. Open VS Code to your CEA LABS folder instead, this genuinely exists already. Step 1 below builds the actual project folder, and re-opens VS Code pointed right at it once it's real.

```
code "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS"  
# Open the integrated terminal with Ctrl + ` (backtick)  
# Every command in this SOP runs inside that panel, PowerShell on Windows
```

## Avoiding the glued-together command problem

Every command in this SOP sits in its own separate gray block, one command per block. When copying, click once inside a single line so you only select that line. If two commands ever get pasted as one and run together, you'll see an error mentioning a flag or word that doesn't exist, that's the sign two lines got glued together.

## Step 1: Create Your Project Folder

```
cd "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS"  
New-Item -ItemType Directory -Path "C:\Users\Gguerra\Documents\Giovanni\REPOS\  
CLOUD ENGINEER\CEA LABS\LAB01" -Force  
cd "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01"  
New-Item -ItemType Directory -Path "C:\Users\Gguerra\Documents\Giovanni\REPOS\  
CLOUD ENGINEER\CEA LABS\LAB01\static-website-lab"  
cd "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\  
static-website-lab"
```

The project folder is real now. Re-open VS Code pointed directly at it, this gives you a clean Explorer panel showing only this lab's files, instead of the whole CEA LABS tree:

```
code "C:\Users\Gguerra\Documents\Giovanni\REPOS\CLOUD ENGINEER\CEA LABS\LAB01\  
static-website-lab"
```

A new VS Code window opens. Open its integrated terminal too (Ctrl + `), and confirm you're actually there before continuing:

```
Get-Location
```

That should show the static-website-lab folder. Every command from Step 2 onward runs in this window, this terminal.

## Step 2: Set Your Lab Variables

Storage account names must be globally unique across every Azure customer on the planet, all lowercase, letters and numbers only, no dashes, no spaces, 3 to 24 characters. Pick something that includes your name and a few random digits so it doesn't collide with someone else's.

**NOTE:** These variables only exist for the current terminal session. If you close VS Code and reopen it later, you'll need to set them again before continuing. That's expected, not a bug.

```
$rg = "rg-lab01-giovanni"  
$location = "eastus"  
$storage = "stlab01giovanni47" # change the digits if this name is already  
taken later
```

Confirm all three actually got set:

```
echo $rg  
echo $location  
echo $storage
```

## Step 3: Create the Resource Group

```
az group create --name $rg --location $location
```

Confirm it actually exists before moving on:

```
az group show --name $rg --query name -o tsv
```

## Step 4: Create the Storage Account

This one command does more than the original portal walkthrough, since it also locks down two important security settings the portal defaults leave open: HTTPS-only transfer, and a modern minimum TLS version.

```
az storage account create `
  --name $storage `
  --resource-group $rg `
  --location $location `
  --sku Standard_LRS `
  --kind StorageV2 `
  --https-only true `
  --min-tls-version TLS1_2 `
  --allow-blob-public-access true
```

**NOTE:** --kind must be StorageV2, general purpose v2. The static website feature does not exist on the older, legacy storage account kinds. --https-only true blocks any plain HTTP request to the storage account's data APIs. --min-tls-version TLS1\_2 refuses connections using older, weaker encryption. --allow-blob-public-access true is required for the site to actually be viewable, Azure's newer accounts default this to false for security, which silently breaks static website hosting if left unset. This only makes it possible for a container to be public, it does not make the whole account public by itself. Enabling static website hosting in Step 5 configures only the \$web container for anonymous read, nothing else in the account is affected.

If the name is rejected as already taken, that's expected, storage names are unique across all of Azure, not just your account:

```
$storage = "stlab01giovanni" + (Get-Random -Maximum 9999)
echo $storage
```

Then re-run the az storage account create command above with the new name.

## Step 5: Enable Static Website Hosting

This single command replaces the entire portal blade the original lab walks through, the toggle, the index document field, and the error document field, all at once.

```
az storage blob service-properties update `
  --account-name $storage `
  --static-website `
  --index-document index.html `
  --404-document 404.html `
  --auth-mode login
```

Capture your website's real public address into a variable so you don't have to retype it later:

```
$endpoint = az storage account show --name $storage --query
"primaryEndpoints.web" -o tsv
echo $endpoint
```

That should print something like <https://stlab01giovanni47.z13.web.core.windows.net/>. That's your website's address, and it's already HTTPS by default, Microsoft issues and manages the certificate for that domain automatically, no extra setup needed.

## Step 6: Create Your Website Files

Two files this time instead of one, the actual page, and a proper error page for anything that doesn't exist, which is genuinely good practice, not just an optional extra.

### index.html

```
New-Item -ItemType File -Path "index.html"  
code index.html
```

Paste the block below into the empty tab that opens, then save with Ctrl+S.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>My First Cloud Site</title>  
  <style>  
    body {  
      font-family: sans-serif;  
      text-align: center;  
      margin-top: 50px;  
      background-color: #f0f0f0;  
    }  
    h1 {  
      color: #0078d4;  
    }  
  </style>  
</head>  
<body>  
  <h1>Hello from the Cloud!</h1>  
  <p>This site is hosted on Azure Blob Storage.</p>  
  <p>Deployed by: Giovanni Guerra</p>  
</body>  
</html>
```

### 404.html

```
New-Item -ItemType File -Path "404.html"  
code 404.html
```

Paste the block below into the empty tab that opens, then save with Ctrl+S.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Page Not Found</title>  
  <style>  
    body {  
      font-family: sans-serif;  
      text-align: center;  
      margin-top: 50px;  
      background-color: #f0f0f0;  
    }  
    h1 {  
      color: #c62828;  
    }  
  </style>  
</head>
```

```
<body>
  <h1>404, Page Not Found</h1>
  <p>The page you're looking for doesn't exist.</p>
  <p><a href="/">Go back home</a></p>
</body>
</html>
```

## Step 7: Upload Your Files

**WARNING:** The container that static website hosting uses is literally named \$web, and PowerShell treats a dollar sign as the start of a variable name. If you type \$web in double quotes, PowerShell will try to substitute a variable called web that doesn't exist, silently turning it into an empty value, and your upload will fail or go to the wrong place. Every command below wraps \$web in single quotes specifically to stop PowerShell from touching it. Do not change the quote style.

```
az storage blob upload `
  --account-name $storage `
  --container-name '$web' `
  --name index.html `
  --file .\index.html `
  --auth-mode login
az storage blob upload `
  --account-name $storage `
  --container-name '$web' `
  --name 404.html `
  --file .\404.html `
  --auth-mode login
```

Confirm both files actually made it into the container:

```
az storage blob list `
  --account-name $storage `
  --container-name '$web' `
  --auth-mode login `
  --query "[].name" -o tsv
```

That should print exactly two lines, index.html and 404.html.

## Step 8: Validate

Start-Process \$endpoint

That opens your default browser straight to your live site. You should see Hello from the Cloud! on a light gray background.

Also worth confirming from the terminal directly, this checks the site returns a real success response instead of just trusting what the browser shows:

```
Invoke-WebRequest -Uri $endpoint -UseBasicParsing | Select-Object StatusCode
```

StatusCode should read 200. That's the standard code for a successful page load.

## Making Sure the Site Is Actually Secure

A public website is supposed to be public, that's the point, but public content and an insecure storage account are two very different things. Here's exactly what is and isn't exposed, and what to double check.

| Setting                           | Why it matters                                                                                                                                                                                       |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTPS only, already set in Step 4 | Blocks any plain, unencrypted HTTP request to the storage account's data APIs. Your actual website endpoint is HTTPS by default regardless, this setting closes the door on the management side too. |

|                                        |                                                                                                                                                                                                                                                              |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Minimum TLS 1.2, already set in Step 4 | Refuses connections attempting to use older, weaker encryption standards that have known vulnerabilities.                                                                                                                                                    |
| Only the \$web container is public     | Enabling static website hosting only exposes that one specific container for anonymous read access. Any other container you create in this same storage account stays private by default.                                                                    |
| Never share your storage account key   | The uploads in this SOP use --auth-mode login, your own Azure AD identity, instead of the storage account's master key. That key grants full read and write access to everything in the account. Losing it is far worse than anyone seeing your public HTML. |
| Enable blob soft delete                | Protects against an accidental overwrite or delete of your site's files. Recoverable for a set number of days instead of gone immediately.                                                                                                                   |

Turn on soft delete now, it costs nothing extra at this scale and genuinely saves you from a bad mistake later:

```
az storage account blob-service-properties update `
  --account-name $storage `
  --resource-group $rg `
  --enable-delete-retention true `
  --delete-retention-days 7`
```

## Optional upgrades if this ever needs to be a real production site

| Upgrade                        | What it adds                                                                                                                                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Azure CDN                      | A custom domain name instead of the long default address, HTTPS on that custom domain, and cached content served from locations closer to visitors around the world                                      |
| Azure Front Door with WAF      | Everything CDN offers, plus a Web Application Firewall filtering malicious traffic before it ever reaches your storage account. Meaningful for a business site, likely overkill for a personal portfolio |
| Microsoft Defender for Storage | Active threat detection on the storage account itself, alerts on unusual access patterns                                                                                                                 |

## Cost Analysis: Running This Full Time

This is a real breakdown, not a rough guess. Static website hosting on Blob Storage bills on three things: how much you store, how many read and write operations happen, and how much data leaves Azure to reach visitors, called egress.

| Cost driver                     | Typical rate                           | What that means here                                                                                                       |
|---------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Storage capacity, Hot tier, LRS | About \$0.018 per GB per month         | A simple HTML and CSS site is a few hundred KB. This cost is effectively pennies, well under \$0.01 a month                |
| Write operations                | About \$0.05 per 10,000 operations     | You only write when you update the site. Even frequent updates stay near zero cost                                         |
| Read operations                 | About \$0.004 per 10,000 operations    | Each page view is a handful of read operations. Tens of thousands of visits still costs cents                              |
| Data egress, outbound bandwidth | About \$0.087 per GB, first 100GB tier | This is the real cost driver. A 500KB page loaded 10,000 times in a month is about 5GB of egress, roughly \$0.40 to \$0.50 |

### Realistic monthly totals:

| Traffic level                                    | Estimated monthly visits | Estimated monthly cost |
|--------------------------------------------------|--------------------------|------------------------|
| Personal portfolio, light traffic                | Under 1,000              | Under \$0.50           |
| Moderate traffic, shared on LinkedIn or a resume | 1,000 to 20,000          | \$0.50 to \$2          |
| Higher traffic, actively promoted site           | 20,000 to 100,000        | \$2 to \$10            |

### If you add optional upgrades:

| Upgrade                                 | Typical added cost                                               | Worth it for this lab?                                                                                                               |
|-----------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Azure CDN, custom domain and caching    | Roughly \$0.081 per GB egress, no fixed monthly fee              | Reasonable if you want a real domain name instead of the long default address, adds only a few dollars a month at this traffic level |
| Azure Front Door Standard, WAF included | About \$35 a month base fee plus usage                           | Not worth it for a personal portfolio site. This is built for business-critical, higher scale workloads                              |
| Custom domain registration              | About \$10 to \$15 a year, paid to a domain registrar, not Azure | Optional, only needed if you want yourname.com instead of the default Azure address                                                  |

**NOTE:** Bottom line: running this exact site full time, indefinitely, at realistic personal traffic levels costs well under \$5 a month, most likely under \$1. The number one thing that would meaningfully increase cost is a sudden spike in traffic driving up egress, not the storage itself.

## Troubleshooting

| Problem                                                              | Cause                                                                                                  | Solution                                                                                                                                                                                                               |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Upload succeeds but the site 404s, or the upload seems to go nowhere | \$web got interpreted as an empty PowerShell variable instead of the literal container name            | Confirm the container name in your command is wrapped in single quotes, '\$web', not double quotes or no quotes at all                                                                                                 |
| az storage account create fails, name already taken                  | Storage account names are unique across all of Azure, not just your subscription                       | Add random digits to the name, an example command for this is in Step 4, then retry with the new name                                                                                                                  |
| Site loads but shows 403 Forbidden instead of the page               | allowBlobPublicAccess is false on the storage account, this is Azure's newer default for new accounts  | Run az storage account update --name \$storage --resource-group \$rg --allow-blob-public-access true, then reload the page                                                                                             |
| 404, The requested content does not exist                            | The file wasn't named exactly index.html, Azure is case-sensitive, or it landed in the wrong container | Re-run the blob list command from Step 7 and confirm the exact file names it shows                                                                                                                                     |
| az storage blob upload fails with an authorization error             | Your account doesn't have the Storage Blob Data Contributor role on this storage account yet           | Assign yourself that role: az role assignment create --assignee (az account show --query user.name -o tsv) --role "Storage Blob Data Contributor" --scope (az storage account show --name \$storage --query id -o tsv) |
| Sign-in page says an account doesn't exist, even though it's         | The browser routed to the personal consumer Microsoft sign-in page                                     | Use az login --tenant YOUR_TENANT_ID --use-device-code,                                                                                                                                                                |



|                                                                  |                                                               |                                                                                   |
|------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| real                                                             | instead of the organization's page                            | and complete it in a fresh incognito window                                       |
| Site shows an old version after re-uploading a change            | Browser cached the previous version locally                   | Hard refresh the page, Ctrl+F5 on Windows, or open it in a fresh incognito window |
| Command fails with an unexpected word or flag that doesn't exist | Two separate commands got pasted and ran together as one line | Check the command against this SOP, each one belongs in its own block             |

## Clean Up Resources

Always clean up a lab environment when you're done with it, keeps your subscription tidy and avoids any lingering cost.

```
az group delete --name $rg --yes --no-wait
```

Confirm it's actually gone:

```
az group show --name $rg
# Expected: ResourceGroupNotFound
```

## Bonus: Words Worth Knowing

| Word                   | What it means, in plain words                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| PaaS                   | Platform as a Service, Azure manages the underlying infrastructure, you only manage your content or code              |
| Serverless             | No server for you to build, patch, or maintain, the cloud provider runs everything underneath                         |
| Blob storage           | Azure's general purpose storage for files of any kind, short for Binary Large Object                                  |
| \$web container        | The specific, special container Azure Storage uses to serve static website content publicly                           |
| Static website hosting | A storage account feature that serves plain HTML, CSS, and JavaScript files directly to the public internet           |
| Endpoint               | The actual public web address Azure gives your storage account once static hosting is enabled                         |
| LRS                    | Locally Redundant Storage, your data is copied three times within a single datacenter, the cheapest redundancy option |
| TLS                    | Transport Layer Security, the encryption protocol behind HTTPS                                                        |
| CDN                    | Content Delivery Network, caches your site at locations around the world so it loads faster for distant visitors      |
| WAF                    | Web Application Firewall, filters out malicious traffic before it reaches your site                                   |
| Egress                 | Data leaving Azure's network, this is what visitors downloading your page actually cost you                           |
| Soft delete            | A safety net that keeps deleted or overwritten files recoverable for a set number of days instead of gone instantly   |